JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA

BEEE

Basic Electrical & Electronics Engineering

Unit – VI

Digital Electronics

Regulation: JNTUK R23 | Semester: I | Branch: Common to All

Comprehensive Study Material | Exam-Oriented | With Diagrams & Q&A

Topics Covered
✓ Number Systems & Conversions
✓ Digital Codes (BCD, Excess-3, Gray, Hamming)
✓ Logic Gates with Truth Tables & SVG Diagrams
✓ Boolean Algebra & Simplification
✓ Combinational Circuits – Half & Full Adder
✓ Sequential Circuits – Flip-Flops, Registers, Counters
✓ 2-Mark & 5-Mark Q&A | Quick Revision Sheet

Section 1

Introduction to Digital Electronics

Digital Electronics is the branch of electronics that deals with digital signals — signals that take only two discrete values: **HIGH (1)** and **LOW (0)**. It forms the foundation of computers, microprocessors, communication systems, and all modern digital devices.

1.1 Analog vs Digital Signals

An **Analog signal** is a continuous signal that varies smoothly over time and can take any value in a range (e.g., audio from a microphone). A **Digital signal** is a discrete signal with only two levels — logic HIGH (1, typically 5V or 3.3V) and logic LOW (0, typically 0V).

✓ Advantages of Digital Systems

- Noise immunity is high
- Easier to store & process information
- More reliable and accurate
- Programmable and flexible
- Cheaper due to VLSI technology

✗ Disadvantages

- Real-world signals are analog; need ADC
- More components sometimes needed
- Quantization error introduced
- Higher power in some designs

1.2 Applications of Digital Electronics

- Computers, laptops, and smartphones
- Digital communication systems (Wi-Fi, 4G/5G)
- Industrial automation and control systems
- Digital cameras, televisions, GPS systems
- ATMs, smart cards, and embedded systems

Key Concept: All digital circuits operate on **Binary (Base-2)** number system using only digits 0 and 1, called **bits**. A group of 8 bits is called a **byte**.

Section 2

Number Systems

A **number system** is a mathematical notation for representing numbers using a set of digits. The **base** or **radix** of a number system defines how many unique digits are available.

Number System	Base/Radix	Digits Used	Example
Decimal	10	0–9	245
Binary	2	0, 1	11110101
Octal	8	0–7	365
Hexadecimal	16	0–9, A–F	F5A

2.1 Decimal Number System (Base-10)

The most commonly used number system in everyday life. Each digit position has a **positional weight** that is a power of 10.

Example: $345 = 3\times10^2 + 4\times10^1 + 5\times10^0 = 300 + 40 + 5 = 345$

2.2 Binary Number System (Base-2)

Used internally by all digital computers. Only two digits: **0** and **1**. Each digit is called a **bit** (Binary digIT). The leftmost bit is the **MSB (Most Significant Bit)** and the rightmost is the **LSB (Least Significant Bit)**.

Example: $(1101)_2 = 1\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0 = 8 + 4 + 0 + 1 = (13)_{10}$

Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Weight	128	64	32	16	8	4	2	1

2.3 Octal Number System (Base-8)

Uses digits 0 to 7. Octal is useful as a shorthand for binary — every octal digit represents exactly **3 binary bits**.

Example: $(725)_8 = 7\times8^2 + 2\times8^1 + 5\times8^0 = 448 + 16 + 5 = (469)_{10}$

2.4 Hexadecimal Number System (Base-16)

Uses digits 0–9 and letters A–F (where A=10, B=11, C=12, D=13, E=14, F=15). Every hex digit represents exactly **4 binary bits** (one nibble). Widely used in memory addresses and color codes.

Example: $(2AF)_{16} = 2\times16^2 + A\times16^1 + F\times16^0 = 2\times256 + 10\times16 + 15\times1 = 512 + 160 + 15 = (687)_{10}$

Decimal	Binary	Octal	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2

3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Section 3

Number System Conversions

3.1 Decimal to Binary Conversion

Method: Repeated division by 2. Divide the number by 2 repeatedly and record remainders from bottom to top.

Convert $(45)_{10}$ to Binary: 45 ÷ 2 = 22 rem 1 ← LSB 22 ÷ 2 = 11 rem 0 11 ÷ 2 = 5 rem 1 5 ÷ 2 = 2 rem 1 2 ÷ 2 = 1 rem 0 1 ÷ 2 = 0 rem 1 ← MSB Result: $(45)_{10} = (101101)_2$

3.2 Binary to Decimal Conversion

Method: Multiply each bit by its positional weight (power of 2) and sum all products.

Convert $(110101)_2$ to Decimal: $= 1\times2^5 + 1\times2^4 + 0\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0 = 32 + 16 + 0 + 4 + 0 + 1 = (53)_{10}$

3.3 Decimal to Octal Conversion

Convert $(156)_{10}$ to Octal: 156 ÷ 8 = 19 rem 4 ← LSB 19 ÷ 8 = 2 rem 3 2 ÷ 8 = 0 rem 2 ← MSB Result: $(156)_{10} = (234)_8$

3.4 Binary to Octal and Octal to Binary

Group binary digits in sets of **3 from right to left**. Each group converts to one octal digit.

Binary to Octal: $(110\ 101\ 011)_2 \rightarrow 6\ 5\ 3 \rightarrow (653)_8$ Octal to Binary: $(472)_8 \rightarrow 4{=}100,\ 7{=}111,\ 2{=}010 \rightarrow (100111010)_2$

3.5 Binary to Hexadecimal and Vice Versa

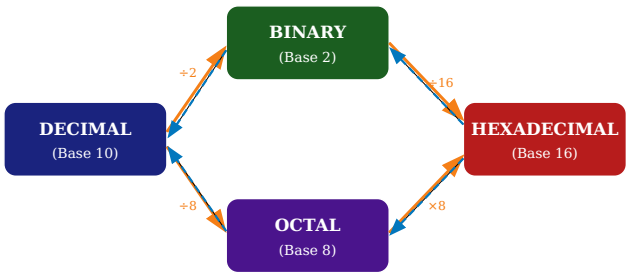
Group binary digits in sets of **4 from right to left**.

Binary to Hex: $(1010\ 1111\ 0011)_2 \rightarrow A\ F\ 3 \rightarrow (AF3)_{16}$ Hex to Binary: $(3B9)_{16} \rightarrow 3{=}0011,\ B{=}1011,\ 9{=}1001 \rightarrow (001110111001)_2$

3.6 Decimal to Hexadecimal

Convert $(255)_{10}$ to Hex: 255 ÷ 16 = 15 rem 15(F) ← LSB 15 ÷ 16 = 0 rem 15(F) ← MSB Result: $(255)_{10} = (FF)_{16}$

Fig 3.1 – Number System Conversion Flowchart



Orange arrows = forward conversion | Blue dashed = reverse conversion

Section 4

Digital Codes

4.1 BCD Code (Binary Coded Decimal)

Definition: BCD is a code in which each decimal digit (0–9) is represented by its 4-bit binary equivalent separately. It uses only the combinations from 0000 to 1001 (16 combinations but only 10 are valid).

Working: Each decimal digit is independently encoded into 4 bits. For example, decimal 29 is not converted as a whole, but digit-by-digit: 2→0010, 9→1001.

```
Example: (93)₁₀ in BCD: 9 → 1001 3 → 0011 BCD = 1001 0011 Reverse: (0110 0101)BCD → 0110=6, 0101=5 → (65)₁₀
```

Advantages

- Easy decimal display
- No conversion needed for display
- Used in calculators

Disadvantages

- Wastes 6 combinations (1010–1111)
- Arithmetic operations complex

4.2 Excess-3 Code (XS-3)

Definition: Excess-3 is a self-complementing BCD code obtained by adding 3 (0011) to each BCD digit. It is a non-weighted code.

```
XS-3 Code = BCD + 0011 Example: Decimal 5 → BCD = 0101 → XS-3 = 0101 + 0011 = 1000 Decimal 9 → BCD = 1001 → XS-3 = 1001 + 0011 = 1100
```

Self-complementing property: The 9's complement of a decimal digit is obtained by inverting (complementing) all bits of its XS-3 code. This is very useful in BCD subtraction.

Decimal	BCD	Excess-3
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001

7	0111	1010
8	1000	1011
9	1001	1100

4.3 Gray Code (Reflected Binary Code)

Definition: Gray code is a non-weighted code in which consecutive numbers differ by only **one bit**. This property is called the **unit distance property**.

Applications: Used in shaft encoders, error detection, K-maps, and digital-to-analog converters to avoid ambiguity during transitions.

```
Binary to Gray Code Conversion: Step 1: MSB of Gray = MSB of Binary Step 2: Each subsequent Gray bit = XOR of current and previous binary bit Example: Binary 1011 → Gray Code G₃ = B₃ = 1 G₂ = B₃ ⊕ B₂ = 1 ⊕ 0 = 1 G₁ = B₂ ⊕ B₁ = 0 ⊕ 1 = 1 G₀ = B₁ ⊕ B₀ = 1 ⊕ 1 = 0 Gray Code = 1110
```

```
Gray Code to Binary Conversion: Step 1: MSB of Binary = MSB of Gray Step 2: Each Binary bit = XOR of previous Binary bit and current Gray bit Example: Gray 1110 → Binary B₃ = G₃ = 1 B₂ = B₃ ⊕ G₂ = 1 ⊕ 1 = 0 B₁ = B₂ ⊕ G₁ = 0 ⊕ 1 = 1 B₀ = B₁ ⊕ G₀ = 1 ⊕ 0 = 1 Binary = 1011 ✓
```

Decimal	Binary	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100

4.4 Hamming Code (Error Detection & Correction)

Definition: Hamming code is an error-detecting and error-correcting code developed by Richard Hamming. It can detect and correct **single-bit errors** and detect (but not correct) **two-bit errors**.

Principle: Parity check bits (P) are inserted at specific positions (powers of 2: 1, 2, 4, 8...) among the data bits. Each parity bit is responsible for checking a specific group of bits.

```
Number of parity bits (r) for m data bits: Condition: 2^r ≥ m + r + 1 For 4 data bits (m=4): 2^r ≥ 4 + r + 1 → r = 3 (since 2³=8 ≥ 8) Total bits = 4 + 3 = 7 bits Parity bit positions: 1(P₁), 2(P₂), 4(P₃) Data bit positions: 3(d₁), 5(d₂), 6(d₃), 7(d₄)
```

Parity bit calculation (even parity):

- P_1 checks positions: 1, 3, 5, 7

- P_2 checks positions: 2, 3, 6, 7
- P_3 checks positions: 4, 5, 6, 7

Section 5

Logic Gates

Logic Gates are the fundamental building blocks of all digital circuits. They implement Boolean functions by performing logical operations on one or more binary inputs to produce a single binary output.

5.1 AND Gate

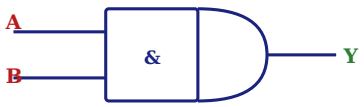
Definition: The AND gate outputs HIGH (1) only when ALL its inputs are HIGH (1).

Boolean Expression:

```
Y = A · B    (also written as Y = AB)
```

Working: The output is the logical product of all inputs. If any single input is 0, the output is 0. Only when every input is 1 does the output become 1.

Fig 5.1 – AND Gate Symbol

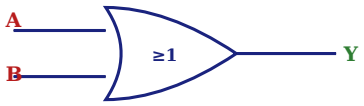


5.2 OR Gate

Definition: The OR gate outputs HIGH (1) when ANY ONE or more of its inputs is HIGH (1). Output is LOW only when all inputs are LOW.

```
Y = A + B
```

Fig 5.2 – OR Gate Symbol

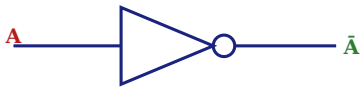


5.3 NOT Gate (Inverter)

Definition: The NOT gate has only one input and one output. It produces the complement (inverse) of its input. If input is 1, output is 0 and vice versa.

```
Y = A'   or   Y = Ā   (A-bar)
```

Fig 5.3 – NOT Gate (Inverter) Symbol



Section 6

Truth Tables of Basic Gates

6.1 AND Gate Truth Table

A	B	Y = A·B
0	0	0
0	1	0
1	0	0
1	1	1

6.2 OR Gate Truth Table

A	B	Y = A+B
0	0	0
0	1	1
1	0	1
1	1	1

6.3 NOT Gate Truth Table

A	Y = Ā
0	1
1	0

Gate Summary

Gate	Symbol	Expression	Output 1 when
AND	&	A·B	All inputs = 1
OR	≥1	A+B	Any input = 1
NOT	▷○	Ā	Input = 0

Section 7

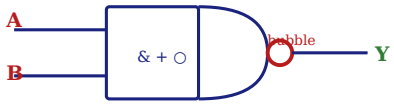
Universal Gates & Special Gates

7.1 NAND Gate

Definition: NAND (NOT-AND) gate produces the complement of AND operation. Its output is LOW only when ALL inputs are HIGH, otherwise the output is HIGH. It is called a **Universal Gate** because any other logic gate can be implemented using NAND gates only.

```
Y = (A · B)' = (AB)'
```

Fig 7.1 – NAND Gate Symbol



NAND Gate Truth Table

A	B	A·B	Y = (AB)'
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

NAND as Universal Gate – Implementing Basic Gates

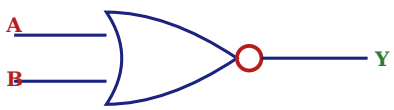
```
NOT using NAND: Y = (A·A)' = A' AND using NAND: Y = ((AB)')' = AB OR using NAND: Y =
(A')·(B') = A+B [using De Morgan's]
```

7.2 NOR Gate

Definition: NOR (NOT-OR) gate produces the complement of OR operation. Output is HIGH only when ALL inputs are LOW. It is also a **Universal Gate**.

```
Y = (A + B)' = (A+B)'
```

Fig 7.2 – NOR Gate Symbol



NOR Gate Truth Table

A	B	A+B	Y = (A+B)'
0	0	0	1

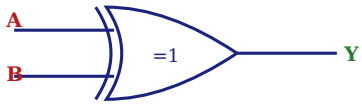
0	1	1	0
1	0	1	0
1	1	1	0

7.3 XOR Gate (Exclusive OR)

Definition: XOR gate outputs HIGH when the inputs are **different** (one is 0, other is 1). Output is LOW when inputs are the same.

```
Y = A ⊕ B = A'B + AB'
```

Fig 7.3 – XOR Gate Symbol



XOR Gate Truth Table

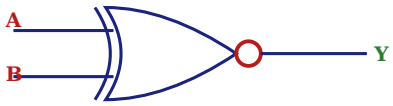
A	B	Y = A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

7.4 XNOR Gate (Exclusive NOR)

Definition: XNOR gate outputs HIGH when both inputs are **equal** (both 0 or both 1). It is the complement of XOR.

```
Y = (A ⊕ B)' = AB + A'B'
```

Fig 7.4 – XNOR Gate Symbol



XNOR Gate Truth Table

A	B	Y = (A⊕B)'
0	0	1
0	1	0
1	0	0
1	1	1

1	1	1

Section 8

Boolean Algebra

Boolean Algebra is a mathematical system that deals with binary variables and logical operations. Introduced by **George Boole** in 1854. It forms the theoretical foundation for digital circuit analysis and design.

8.1 Basic Laws of Boolean Algebra

Law	AND Form	OR Form
Identity Law	A · 1 = A	A + 0 = A
Null Law	A · 0 = 0	A + 1 = 1
Idempotent Law	A · A = A	A + A = A
Complement Law	A · A' = 0	A + A' = 1
Double Complement	(A')' = A	
Commutative Law	A · B = B · A	A + B = B + A
Associative Law	A(BC) = (AB)C	A+(B+C)=(A+B)+C
Distributive Law	A(B+C)=AB+AC	A+BC=(A+B)(A+C)
Absorption Law	A(A+B) = A	A + AB = A

8.2 De Morgan's Theorems

De Morgan's Theorems are the two most important theorems in Boolean algebra used for complementing expressions and simplification.

```
Theorem 1: (A · B)' = A' + B' "Complement of a product = Sum of complements" Theorem 2: (A + B)' = A' · B' "Complement of a sum = Product of complements" Example: Simplify (AB + C)' = (AB)' · C' [Theorem 2] = (A' + B') · C' [Theorem 1]
```

8.3 Simplification of Boolean Expressions

Example 1: Simplify Y = AB + AB'

```
Y = AB + AB' = A(B + B') [Distributive Law] = A · 1 [Complement Law] = A [Identity Law]
```

Example 2: Simplify Y = A + A'B

```
Y = A + A'B = (A + A')(A + B) [Distributive Law] = 1 · (A + B) [Complement Law] = A + B [Identity Law]
```

Example 3: Simplify Y = ABC + ABC' + AB'C

```
Y = ABC + ABC' + AB'C = AB(C + C') + AB'C [Factoring AB] = AB(1) + AB'C [Complement] = AB + AB'C [Identity] = A(B + B'C) [Factor A] = A(B + C) [Absorption variant]
```

Section 9

Combinational Circuits – Introduction

Combinational circuits are digital circuits in which the output at any instant of time depends **only on the current input values** and NOT on any previous inputs or outputs. They have NO memory elements (flip-flops).

Key Characteristics

- Output is a **pure function** of present inputs
- No feedback paths
- No clock required
- Examples: Adders, Subtractors, Multiplexers, Decoders, Encoders

Design steps for Combinational Circuits: 1. State the problem clearly → 2. Identify inputs and outputs → 3. Derive truth table → 4. Write Boolean expressions → 5. Simplify using Boolean algebra or K-map → 6. Implement using logic gates

Section 10

Half Adder

Half Adder is a combinational circuit that adds two single-bit binary numbers (A and B) and produces two outputs: **Sum (S)** and **Carry (C)**. It is called "half" adder because it cannot add a carry from a previous stage.

10.1 Truth Table

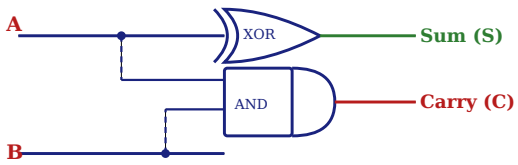
A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

10.2 Boolean Expressions

```
Sum S = A ⊕ B (XOR operation) Carry C = A · B (AND operation)
```

10.3 Logic Circuit Diagram

Fig 10.1 – Half Adder Logic Circuit



10.4 Block Diagram of Half Adder

Fig 10.2 – Half Adder Block Diagram

Advantages

- Simple and fast circuit
- Minimum gate count
- Easy to design

Disadvantages

- Cannot add carry-in

- Not suitable for multi-bit addition alone

Section 11

Full Adder

Full Adder is a combinational circuit that adds three 1-bit binary numbers: two data bits (**A** and **B**) and one carry-in (**C_{in}**). It produces two outputs: **Sum (S)** and **Carry-out (C_{out})**. Full adders can be cascaded to add multi-bit numbers.

11.1 Truth Table

A	B	Cin	Sum (S)	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

11.2 Boolean Expressions

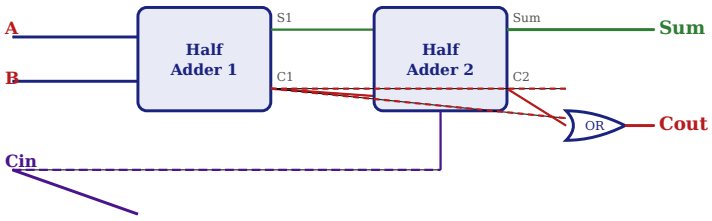
```
Sum S = A ⊕ B ⊕ Cin Cout = A·B + B·Cin + A·Cin = AB + (A⊕B)·Cin [using Half Adder]
```

11.3 Full Adder using Two Half Adders

A Full Adder can be constructed using **two Half Adders and an OR gate**:

- First Half Adder adds A and B → produces intermediate Sum S_1 and Carry C_1
- Second Half Adder adds S_1 and Cin → produces final Sum S and intermediate Carry C_2
- OR gate combines C_1 and C_2 → produces C_{out}

Fig 11.1 – Full Adder using Two Half Adders



Advantages

- Can add three bits including carry-in
- Can be cascaded for n-bit addition
- Foundation of all arithmetic circuits

Disadvantages

- More complex than Half Adder
- More gates required
- Propagation delay in cascaded form

Section 12

Sequential Circuits

Sequential circuits are digital circuits in which the output depends not only on the **present inputs** but also on the **past history** (previous states). They contain memory elements (flip-flops) and require a clock signal for synchronization.

Key Characteristics

- Output = f (present input, present state)
- Next state = g (present input, present state)
- Contain feedback paths and memory elements
- Require clock signal (synchronous) or level-triggered (asynchronous)
- Examples: Flip-Flops, Registers, Counters, Finite State Machines

12.1 Flip-Flops – Introduction

Flip-Flop (FF) is the basic memory element of sequential circuits. It is a bistable multivibrator that can store one bit of information (0 or 1) and changes state based on the input and clock signal. It has two stable states — SET (Q=1) and RESET (Q=0).

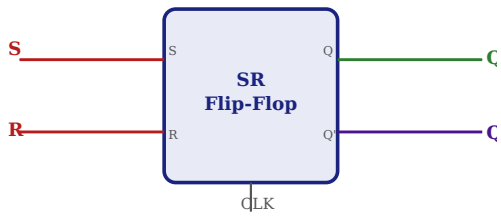
12.2 SR Flip-Flop (Set-Reset)

Definition: SR flip-flop has two inputs — S (Set) and R (Reset) — and two outputs Q and Q'. When S=1,R=0: Q becomes 1 (Set). When S=0,R=1: Q becomes 0 (Reset). When S=R=1: output is invalid/forbidden.

S	R	Q(next)	State
0	0	Q (No change)	Hold
0	1	0	Reset
1	0	1	Set
1	1	Invalid	Forbidden

```
Characteristic Equation of SR FF: Q(next) = S + R'Q Constraint: S · R = 0 (forbidden state)
```

Fig 12.1 – SR Flip-Flop Block Diagram



12.3 JK Flip-Flop

Definition: JK flip-flop is an improved version of SR flip-flop. It eliminates the invalid state by toggling the output when both J=1 and K=1. J input acts like S (Set) and K acts like R (Reset).

J	K	Q(next)	State
0	0	Q	No Change
0	1	0	Reset
1	0	1	Set
1	1	Q'	Toggle

```
Characteristic Equation of JK FF: Q(next) = JQ' + K'Q
```

12.4 D Flip-Flop (Data/Delay)

Definition: D flip-flop has a single input D. The output follows the input D on every active clock edge. It is used as a data register and delay element.

D	Q(next)
0	0
1	1

```
Characteristic Equation of D FF: Q(next) = D
```

12.5 T Flip-Flop (Toggle)

Definition: T flip-flop has a single input T. When T=0, the output holds its current state. When T=1, the output toggles (changes to opposite state) on every clock edge.

T	Q(next)	State
0	Q	No Change
1	Q'	Toggle

```
Characteristic Equation of T FF: Q(next) = T ⊕ Q = TQ' + T'Q
```

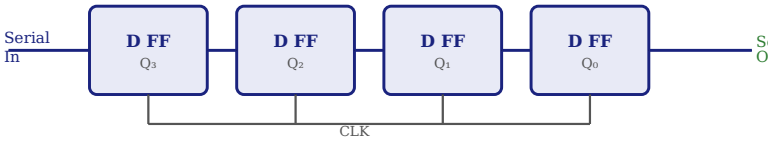
12.6 Registers

Register is a group of flip-flops that can store multiple bits of binary data. An n-bit register contains n flip-flops and can store an n-bit binary word. Registers are used for temporary storage of data in CPUs.

Types of Registers:

- **Serial In – Serial Out (SISO):** Data enters and exits one bit at a time
- **Serial In – Parallel Out (SIPO):** Data enters serially, exits all bits simultaneously
- **Parallel In – Serial Out (PISO):** Data loaded simultaneously, exits one bit at a time
- **Parallel In – Parallel Out (PIPO):** Fastest; data loads and reads simultaneously

Fig 12.2 – 4-Bit Shift Register Block Diagram



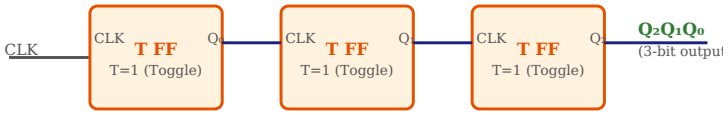
12.7 Counters

Counter is a sequential circuit that counts the number of clock pulses or events. It progresses through a predefined sequence of binary states. An n-bit counter can count from 0 to $(2^n–1)$.

Types of Counters:

- **Asynchronous (Ripple) Counter:** Flip-flops are triggered one after another; each FF's output clocks the next. Simple but slower due to propagation delay.
- **Synchronous Counter:** All flip-flops are clocked simultaneously. Faster and more reliable.
- **Up Counter:** Counts from 0 upward (0,1,2,3...)
- **Down Counter:** Counts downward (...3,2,1,0)
- **Mod-N Counter:** Counts N states (0 to N-1)

Fig 12.3 – 3-Bit Asynchronous Ripple Counter




```
Mod-N Counter uses N flip-flops if 2ⁿ = N. 3-bit counter: 2³ = 8 states (000 to 111) → Mod-8 counter 4-bit counter: 2⁴ = 16 states (0000 to 1111) → Mod-16 counter
```

Section 13

Questions & Answers

Part A – 2 Mark Questions

Q1. Define Binary Number System. 2 Marks

The binary number system is a base-2 positional number system that uses only two digits: **0** and **1**. Each digit in the binary system is called a **bit** (Binary digIT). The positional weights in binary are powers of 2 (...16, 8, 4, 2, 1). Binary is the fundamental number system used internally by all digital computers and electronic devices because it directly maps to the two voltage levels (HIGH and LOW) in digital circuits. A group of 4 bits is called a *nibble* and a group of 8 bits is called a *byte*.

Q2. What is a Logic Gate? Give examples. 2 Marks

A **Logic Gate** is a fundamental building block of digital circuits that performs a logical operation on one or more binary inputs to produce a single binary output. Logic gates implement Boolean functions electronically. The basic logic gates are AND, OR, and NOT gates. The universal gates are NAND and NOR, which can implement any other gate. Special purpose gates include XOR (Exclusive-OR) and XNOR (Exclusive-NOR). Logic gates are constructed using transistors, diodes, and resistors in integrated circuit form and operate with voltage levels representing binary 0 and 1.

Q3. State De Morgan's Theorems. 2 Marks

De Morgan's Theorems are two fundamental Boolean algebra theorems used for complementing expressions:

Theorem 1: The complement of a product (AND) equals the sum (OR) of the complements: **(A·B)' = A' + B'**

Theorem 2: The complement of a sum (OR) equals the product (AND) of the complements: **(A+B)' = A'·B'**

These theorems enable conversion between NAND and NOR implementations and simplification of complex Boolean expressions. They are extensively used in digital circuit design and analysis.

Q4. What is BCD Code? 2 Marks

BCD (Binary Coded Decimal) is a type of binary encoding in which each decimal digit (0–9) is represented individually by its 4-bit binary equivalent. Unlike pure binary conversion of an entire number, BCD encodes each digit separately. For example, decimal 97 in BCD is 1001 0111 (9→1001, 7→0111). BCD uses only 10 of the 16 possible 4-bit combinations (0000 to 1001); the remaining six (1010 to 1111) are invalid or unused. BCD is widely used in digital displays, calculators, and financial applications where decimal representation must be maintained.

Q5. What is the difference between a Half Adder and a Full Adder? 2 Marks

A **Half Adder** adds two single-bit binary numbers (A and B) and produces Sum = A⊕B and Carry = A·B. It has 2 inputs and 2 outputs, but it *cannot* accept a carry input from a previous stage, making it

suitable only for the least significant bit position.

A **Full Adder** adds three bits — two data bits (A and B) plus a carry-in (C_{in}) — and produces Sum = $A \oplus B \oplus C_{in}$ and Carry-out $C_{out} = AB + BC_{in} + AC_{in}$. It has 3 inputs and 2 outputs. Full adders can be cascaded to build multi-bit binary adders, whereas half adders alone cannot achieve this.

Q6. What is a Flip-Flop? Name its types. 2 Marks

A **Flip-Flop** is a bistable sequential logic circuit that serves as the basic memory element in digital systems. It has two stable states (SET = Q=1 and RESET = Q=0) and can store one bit of information. The flip-flop changes its state based on the input signals and the clock trigger.

Types of Flip-Flops: (i) SR Flip-Flop (Set-Reset) — basic FF with forbidden state. (ii) JK Flip-Flop — improved SR with toggle capability. (iii) D Flip-Flop (Data/Delay) — output follows input D. (iv) T Flip-Flop (Toggle) — output toggles when T=1. Each type has distinct characteristic equations and is used in specific applications.

Part B – 5 Mark Questions

Q7. Explain the working of a Half Adder with truth table and neat diagram. 5 Marks

Introduction: A Half Adder is a fundamental combinational arithmetic circuit designed to add two single-bit binary numbers.

Inputs: A, B (two 1-bit binary numbers)
Outputs: Sum (S) and Carry (C)

Truth Table:

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Derivation of Boolean Expressions: From the truth table, Sum is HIGH when inputs differ, which is the XOR pattern. Carry is HIGH only when both inputs are 1, which is the AND pattern.

Boolean Expressions:

```
S = A ⊕ B (XOR gate)
C = A · B (AND gate)
```

Circuit: The Half Adder requires only two gates — one XOR gate for Sum and one AND gate for Carry. Both gates receive the same inputs A and B simultaneously.

Working: When A=1, B=1: S=1⊕1=0 (no sum in single bit), C=1·1=1 (carry generated). This carry must be passed to the next bit position, which is why a Full Adder is needed for cascading.

Application: Used in the LSB position of binary adders, in ALU design, and digital calculators.

Q8. Explain the JK Flip-Flop with truth table, characteristic equation and diagram. 5 Marks

Introduction: The JK Flip-Flop is the most versatile and widely used flip-flop. It was named after Jack Kilby. It overcomes the forbidden state problem of SR Flip-Flop by introducing a toggle operation when both inputs are 1.

Inputs: J (Set equivalent), K (Reset equivalent), CLK (Clock)
Outputs: Q (normal), Q' (complement)

Operation:

- J=0, K=0: No change. Output remains at previous state Q
- J=0, K=1: Reset condition. Q becomes 0 regardless of previous state
- J=1, K=0: Set condition. Q becomes 1 regardless of previous state
- J=1, K=1: Toggle condition. Q changes to its complement Q'

Truth Table:

J	K	Qn	Q(n+1)	Operation
0	0	0	0	No Change
0	0	1	1	No Change
0	1	0	0	Reset
0	1	1	0	Reset
1	0	0	1	Set
1	0	1	1	Set
1	1	0	1	Toggle
1	1	1	0	Toggle

```
Characteristic Equation: Q(n+1) = JQ' + K'Q
```

Applications: Used in counters, shift registers, frequency dividers, and memory circuits. When J and K are connected together (J=K=T), it acts as a T Flip-Flop (toggle), and when J=D, K=D', it functions as a D Flip-Flop.

Q9. Explain the different number systems with conversion methods and examples. 5 Marks

Introduction: A number system is a mathematical framework using symbols (digits) to represent numerical values. Different bases are used for different purposes in computing.

1. Decimal (Base-10): Uses digits 0–9. Used in everyday mathematics. Positional weights are powers of 10. Example: 345 = 3×100 + 4×10 + 5×1.

2. Binary (Base-2): Uses only 0 and 1. Used internally by all computers. Positional weights are powers of 2. Example: $(1101)_2$ = 8+4+0+1 = $(13)_{10}$.

3. Octal (Base-8): Uses digits 0–7. Each octal digit = 3 binary bits. Example: $(73)_8$ = 7×8+3 = $(59)_{10}$.

4. Hexadecimal (Base-16): Uses 0–9 and A–F. Each hex digit = 4 binary bits. Example: $(1F)_{16}$ = 1×16+15 = $(31)_{10}$.

Conversion Methods:

```
Decimal to Binary: Repeated division by 2 Example: 13 → 13÷2=6r1, 6÷2=3r0, 3÷2=1r1, 1÷2=0r1 → (1101)₂ Binary to Decimal: Sum of bit×weight (1101)₂ = 8+4+0+1 = 13 Binary↔Octal: Group 3 bits; (110 101)₂ → (65)₈ Binary↔Hex: Group 4 bits; (0001 1010)₂ → (1A)₁₆
```

Applications: Binary — CPU operations; Octal — Unix file permissions; Hex — memory addresses, HTML colors (#FF5733), machine code representation.

Q10. Explain NAND gate as a Universal Gate with neat diagrams. 5 Marks

Introduction: A **Universal Gate** is one that can be used to implement any Boolean function without requiring any other type of logic gate. The NAND gate is a universal gate because any logic circuit (NOT, AND, OR, NOR, XOR) can be built using NAND gates alone.

NAND Gate: Output Y = (AB)'. Output is LOW only when all inputs are HIGH.

Implementation of Basic Gates using NAND:

NOT from NAND: Connect both inputs of a NAND gate together: Y = (A·A)' = A'. This requires 1 NAND gate.

AND from NAND: First create NAND(A,B), then invert using another NAND: Y = ((AB)')' = AB. This requires 2 NAND gates.

OR from NAND: Invert each input using NAND (as NOT), then NAND those results: Y = (A'·B')' = A+B (by De Morgan's). This requires 3 NAND gates.

```
NOT : Y = (A·A)' = A' [1 NAND] AND : Y = ((A·B)')' = A·B [2 NANDs] OR : Y = (A'·B')' = A+B [3 NANDs]
```

Advantages of Universal Gates: Reduce the number of distinct IC packages needed; simplified manufacturing; any Boolean function implementable with a single gate type; reduces design complexity and cost.

Applications: NAND-based circuits are used in all major digital systems including processors, memory ICs, and programmable logic devices (PLDs, FPGAs).

Section 14

Quick Revision Sheet

⚡ Last-Minute Revision — Unit VI Digital Electronics

Key formulas, identities, truth tables, and important points for exam preparation.

🔢 Number Systems

- Decimal: Base 10, digits 0–9
- Binary: Base 2, digits 0–1
- Octal: Base 8, digits 0–7
- Hex: Base 16, digits 0–9,A–F
- Octal digit = 3 binary bits
- Hex digit = 4 binary bits

🖳 Logic Gate Expressions

- AND: Y = A·B
- OR: Y = A+B
- NOT: Y = A'
- NAND: Y = (AB)'
- NOR: Y = (A+B)'
- XOR: Y = A⊕B
- XNOR: Y = (A⊕B)'

📋 Boolean Laws (Key)

- A+0=A, A·1=A (Identity)
- A+1=1, A·0=0 (Null)
- A+A'=1, A·A'=0 (Complement)
- A+A=A, A·A=A (Idempotent)
- (A')'=A (Double complement)
- (AB)'=A'+B' (De Morgan 1)
- (A+B)'=A'·B' (De Morgan 2)

+ Adder Equations

- **Half Adder:**
- Sum S = A ⊕ B
- Carry C = A·B
- **Full Adder:**
- Sum S = A ⊕ B ⊕ Cin
- Cout = AB + BCin + ACin

🔁 Flip-Flop Equations

- SR FF: Q* = S + R'Q; SR=0
- JK FF: Q* = JQ' + K'Q
- D FF: Q* = D

- T FF: Q* = T⊕Q

Codes Summary

- BCD: 4 bits per decimal digit
- XS-3: BCD + 0011
- Gray: adjacent codes differ by 1 bit
- Hamming: 2^r ≥ m+r+1
- Gray to Binary: XOR chain

All Truth Tables Summary

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

Conversion Quick Reference

```
Decimal → Binary : Divide by 2, read remainders upward Binary → Decimal : Multiply each bit by 2^position, sum all Decimal → Octal : Divide by 8, read remainders upward Binary → Octal : Group 3 bits from right, convert each Binary → Hex : Group 4 bits from right, convert each BCD → Decimal : Convert each 4-bit group separately XS-3 code : Add 3 (0011) to each BCD digit Gray to Binary : MSB same; each bit = XOR(prev binary, curr gray)
```

Important Points to Remember

- NAND and NOR are called **Universal Gates** — they can implement any Boolean function
- XOR gate output = 1 when inputs are *different*; XNOR = 1 when inputs are *same*
- Combinational circuits have **no memory**; Sequential circuits **have memory** (flip-flops)
- JK Flip-Flop eliminates the **forbidden state** of SR Flip-Flop using the Toggle mode
- A Half Adder uses **XOR + AND**; Full Adder = Two Half Adders + OR gate
- Gray code has **unit distance property** — adjacent codes differ by exactly 1 bit
- Hamming code can **detect and correct** single-bit errors in transmitted data
- An n-bit counter has **2^n states** and is called a Mod-2^n counter
- D Flip-Flop: **Q* = D** — the simplest; acts as a 1-bit memory / data latch
- T Flip-Flop (Toggle) is used in **counters and frequency dividers**

Exam Tips:

- Always draw neat labeled diagrams — they carry marks
- Write Boolean expressions before explaining circuits
- In conversions, show step-by-step working for full marks
- Mention "universal gate" when asked about NAND/NOR
- For flip-flop questions, always include characteristic equation